```
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>  py -3.13
test_autograd_integration.py

================================================================================
🚀 ЗАПУСК ИНТЕГРАЦИОННОГО ТЕСТА: ПЕРЕДАЧА ГРАДИЕНТОВ ИЗ
LOCALRUNNER В LINEAR
================================================================================
Тестирование на устройстве: NVIDIA GeForce RTX 2060

[autograd_debug]добавлен элемент в стек обратного прохода!
[compiler_debug]строка полсе пред обработки (до выявления токенов)='gate_A =
nn_sigmoid(gate_vectors)
'
[debug_compiler]для строки 'gate_A = nn_sigmoid(gate_vectors)
' найденной 2 тензоров
[debug_compiler_log]результат компиляции строки:'if (tid < gate_vectorsSize) { gate_A[tid] =
nn_sigmoid(gate_vectors[tid]); }
'

[compiler_debug]строка полсе пред обработки (до выявления токенов)='memory_state =
memory_state * gate_A
'
[debug_compiler]для строки 'memory_state = memory_state * gate_A
' найденной 3 тензоров
[debug_compiler_log]результат компиляции строки:'if (tid < gate_vectorsSize)
{ memory_state[tid] = memory_state[tid] * gate_A[tid]; }
'

[compiler_debug]строка полсе пред обработки (до выявления токенов)='output = memory_state
+ gate_A
'
[debug_compiler]для строки 'output = memory_state + gate_A
' найденной 4 тензоров
[debug_compiler_log]результат компиляции строки:'if (tid < gate_vectorsSize) { output[tid] =
memory_state[tid] + gate_A[tid]; }
'

[debug_log] ядро прямого прохода к выполнению:
'if (tid < gate_vectorsSize) { gate_A[tid] = nn_sigmoid(gate_vectors[tid]); }

    if (tid < gate_vectorsSize) { memory_state[tid] = memory_state[tid] * gate_A[tid]; }

    if (tid < gate_vectorsSize) { output[tid] = memory_state[tid] + gate_A[tid]; }
'

C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\pyopencl\
__init__.py:581: CompilerWarning: Non-empty compiler output encountered. Set the environment
variable PYOPENCL_COMPILER_OUTPUT=1 to see more.
  lambda: self._prg.build(options_bytes, devices),
[debug_log]прямой проход завершен!


[autograd_debug] СТАРТ BACKWARD. Всего задач в стеке: 1
C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_glob.py:134:
RepeatedKernelRetrieval: Kernel 'bmm_tiled' has been retrieved more than once. Each retrieval
creates a new, independent kernel, at possibly considerable expense. To avoid the expense, reuse the
retrieved kernel instance. To avoid this warning, use cl.Kernel(prg, name).
```

```
  self.prg_matmul.bmm_tiled(
[autograd_debug] Итерация завершена! Задач в стеке: 0
'[[0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]]'


Вычисление параллельного эталона на PyTorch для сверки...

Сверка результатов градиентов весов:
✖ ОШИБКА: Обнаружено расхождение в градиентах весов!
Traceback (most recent call last):
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_autograd_integration.py",
line 142, in <module>
    test_skvoznoi_autograd_pipeline()
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_autograd_integration.py",
line 139, in test_skvoznoi_autograd_pipeline
    raise e
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_autograd_integration.py",
line 134, in test_skvoznoi_autograd_pipeline
    np.testing.assert_allclose(cl_grad_w_linear, pytorch_grad_w_linear, rtol=1e-4, atol=1e-4)

~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^
  File "C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\numpy\
testing\_private\utils.py", line 1780, in assert_allclose
    assert_array_compare(compare, actual, desired, err_msg=str(err_msg),
    ~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                         verbose=verbose, header=header, equal_nan=equal_nan,
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                         strict=strict)
                         ^^^^^^^^^^^^^^
  File "C:\Users\Дмитрий\AppData\Local\Programs\Python\Python313\Lib\site-packages\numpy\
testing\_private\utils.py", line 988, in assert_array_compare
    raise AssertionError(msg)
AssertionError:
Not equal to tolerance rtol=0.0001, atol=0.0001

Mismatched elements: 64 / 64 (100%)
First 5 mismatches are at indices:
 [0, 0]: 0.0 (ACTUAL), -0.7222121357917786 (DESIRED)
 [0, 1]: 0.0 (ACTUAL), 0.40519219636917114 (DESIRED)
 [0, 2]: 0.0 (ACTUAL), -0.8504490256309509 (DESIRED)
 [0, 3]: 0.0 (ACTUAL), -0.014524519443511963 (DESIRED)
 [0, 4]: 0.0 (ACTUAL), 0.4766317903995514 (DESIRED)
Max absolute difference among violations: 1.332085
Max relative difference among violations: 1.
 ACTUAL: array([[0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0., 0., 0.],...
 DESIRED: array([[-0.722212,  0.405192, -0.850449, -0.014525,  0.476632, -1.307686,
        -0.295288,  0.18387 ],
       [-0.677101,  0.334779, -0.865327,  0.052138,  0.336522, -1.054247,...
```

PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > core > autograd.py

```python
import pyopencl as cl
IS_DEBUG=True
class AutogradContext:
    """
    Единый менеджер графа вычислений, привязанный к конкретной очереди OpenCL.
    Устраняет разрозненность стеков у разных тензоров.
    """
    def __init__(self, queue: cl.CommandQueue):
        self.queue = queue
        self.tape = []          # Единая лента (стек) для backward-колбэков
        self.is_training = True # Флаг режима обучения (запись в tape)

    def register_backward(self, backward_fn):
        """Добавляет операцию в общую ленту, если включен режим обучения"""
        if self.is_training:
            self.tape.append(backward_fn)
            if IS_DEBUG:
                print(f"\n[autograd_debug] добавлен элемент в стек обратного прохода!")

    def run_backward(self):
        """Последовательно разворачивает всю ленту от конца к началу с защитой от рекурсии"""
        print(f"\n[autograd_debug] СТАРТ BACKWARD. Всего задач в стеке: {len(self.tape)}")

        # КРИТИЧЕСКИЙ ПРЕДОХРАНИТЕЛЬ: временно отключаем запись новых операций в режиме backward
        self.is_training = False

        try:
            while self.tape:
                current_len = len(self.tape)
                backward_fn = self.tape.pop()
                backward_fn()

                if IS_DEBUG:
                    print(f"[autograd_debug] Итерация завершена! Задач в стеке: {len(self.tape)}")
        finally:
            # Возвращаем режим обучения обратно по завершении прохода
            self.is_training = True

        self.queue.finish()



    def zero_grad(self, params):
        """Безопасно зануляет буферы градиентов для списка тензоров"""
        for param in params:
            #param.requires_grad()
            if param.grad_buffer is not None:
                cl.enqueue_fill_buffer(
                    self.queue,
                    param.grad_buffer,
                    param.dtype.type(0.0),
                    0,
                    param.nbytes
                )
        self.queue.finish()
```

```python
import functools
from core.tensor import Tensor

def auto_backward(func):
    """
    Декоратор для автоматической регистрации графа.
    Ищет парный метод с суффиксом '_backward' и добавляет его в стек контекста.
    """
    @functools.wraps(func)
    def wrapper(self, *args, **kwargs):
        # 1. Выполняем оригинальный метод (например, bmm или softmax)
        out_tensor = func(self, *args, **kwargs)

        # Если autograd отключен в модели (например, при eval/inference), просто возвращаем результат
        if hasattr(self, 'autograd_ctx') and not self.autograd_ctx.is_training:
            return out_tensor

        # 2. Проверяем, требуют ли входные тензоры расчета градиентов
        input_tensors = [arg for arg in args if isinstance(arg, Tensor)]
        requires_grad = any(t.grad_buffer is not None for t in input_tensors)

        if requires_grad:
            # Выделяем буфер градиента для выходного тензора, если его нет
            out_tensor.require_grad()

            # Находим парный backward метод по имени (например, для 'bmm' -> 'bmm_backward')
            backward_method_name = f"{func.__name__}_backward"
            backward_method = getattr(self, backward_method_name, None)

            if backward_method is not None:
                # Создаем замыкание (колбэк), которое будет вызвано во время run_backward()
                def backward_closure():
                    # Автоматически вызываем: метод_backward(grad_output, *входные_тензоры)
                    backward_method(out_tensor, *args, **kwargs)

                # Записываем это действие в единую очередь нашей модели!
                if hasattr(self, 'autograd_ctx'):
                    self.autograd_ctx.register_backward(backward_closure)

        return out_tensor
    return wrapper
```

глобапльные объекты с атрибутом работают некорректно!
Я предлагю создать в классе AutogradContext статический сингелтон и(паттерн одиночка) и его брать из атрибутов. Что об этом думаешь?